Recursion

Repeat Me Smaller!

CS 101: Programming for Future Advanced Engineers

Mastering Recursion in 45 Minutes

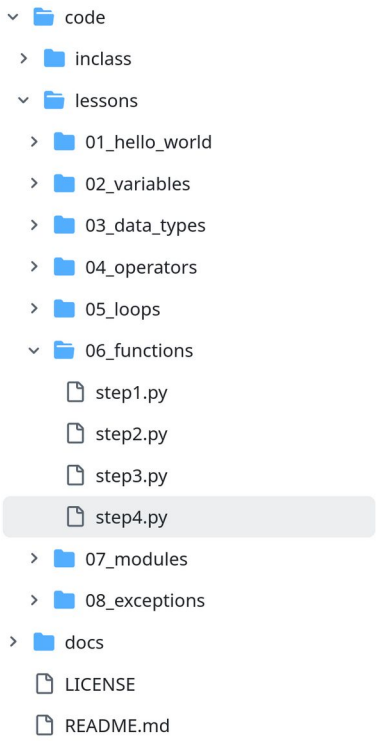
- Recap
- What Is Recursion?
- Why Use Recursion?
- Think Recursively
- When to **Avoid** Recursion
- When to **Use** Recursion

Big problem → Smaller problem → Even smaller problem

Recap

https://github.com/amirih/cs-101-2026-spring

- We are in **step4.py** of the **06_functions** folder




```python
# recursion examples
# recursion is a programming technique where a function calls itself in order to
# solve a problem. It typically involves a base case that stops the recursion and
# a recursive case that breaks the problem into smaller subproblems.

def sum_to_n(n):
    # Base case: if n is 1, return 1
    if n == 1:
        return 1
    else:
        # Recursive case: return n plus the sum of the first n-1 natural numbers
        return n + sum_to_n(n - 1)

def factorial(n):
    # Base case: if n is 0, return 1
    if n == 0:
        return 1
    else:
        # Recursive case: return n multiplied by the factorial of n-1
        return n * factorial(n - 1)

def fibonacci(n):
    # Base case: if n is 0, return 0
    if n == 0:
        return 0
    # Base case: if n is 1, return 1
    elif n == 1:
        return 1
    else:
        # Recursive case: return the sum of the two preceding Fibonacci numbers
        return fibonacci(n - 1) + fibonacci(n - 2)
```

Recap

- A **function** or **method** is a reusable block of code that performs a specific task
- It can receive inputs called **parameters** or **arguments**
- A function may **return** a value, or it may simply perform an action

```python
def print_hello():
    print("Hello, world!")

if __name__ == '__main__':

    print_hello()
```

Recap

```python
def with_parameters_no_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")

def with_parameters_with_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")
    return a + b + c

def no_parameters_no_return():
    print("This function has no parameters")
    print("and it does not return anything.")

def no_parameters_with_return():
    print("This function has no parameters")
    print("but it returns a message.")
    return "Hello, World!"
```

```python
if __name__ == '__main__':

    with_parameters_no_return(1, 2, 3)

    result = with_parameters_with_return(1, 2, 3)
    print(f"Result: {result}")

    no_parameters_no_return()

    message = no_parameters_with_return()
    print(f"Message: {message}")
```

Recap

```python
def with_parameters_no_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")

def with_parameters_with_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")
    return a + b + c

def no_parameters_no_return():
    print("This function has no parameters")
    print("and it does not return anything.")

def no_parameters_with_return():
    print("This function has no parameters")
    print("but it returns a message.")
    return "Hello, World!"
```

```python
if __name__ == '__main__':

    with_parameters_no_return(1, 2, 3)

    result = with_parameters_with_return(1, 2, 3)
    print(f"Result: {result}")

    no_parameters_no_return()

    message = no_parameters_with_return()
    print(f"Message: {message}")
```

Recap

```python
def with_parameters_no_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")

def with_parameters_with_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")
    return a + b + c

def no_parameters_no_return():
    print("This function has no parameters")
    print("and it does not return anything.")

def no_parameters_with_return():
    print("This function has no parameters")
    print("but it returns a message.")
    return "Hello, World!"
```

```python
if __name__ == '__main__':

    with_parameters_no_return(1, 2, 3)

    result = with_parameters_with_return(1, 2, 3)
    print(f"Result: {result}")

    no_parameters_no_return()

    message = no_parameters_with_return()
    print(f"Message: {message}")
```

Recap

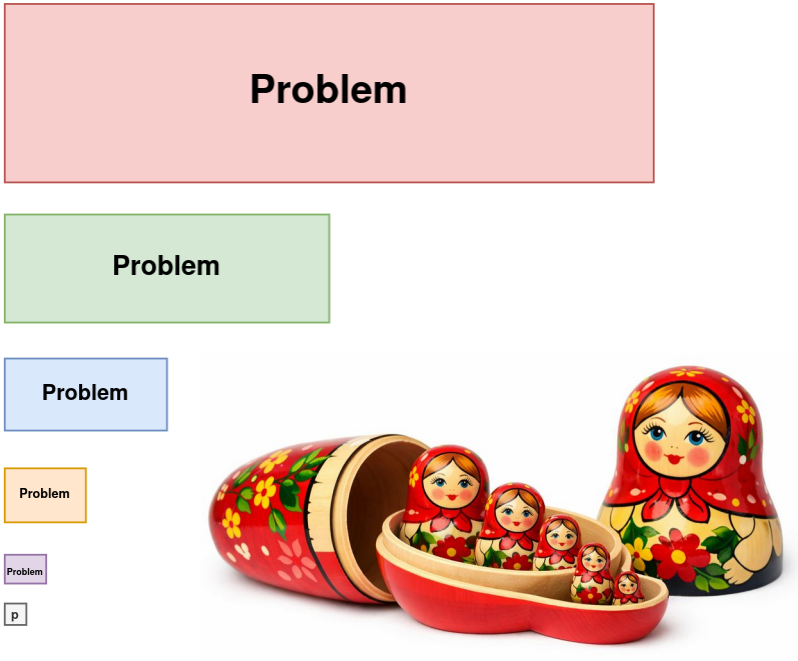
```python
def with_parameters_no_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")

def with_parameters_with_return(a, b, c):
    print(f"a: {a}, b: {b}, c: {c}")
    return a + b + c

def no_parameters_no_return():
    print("This function has no parameters")
    print("and it does not return anything.")

def no_parameters_with_return():
    print("This function has no parameters")
    print("but it returns a message.")
    return "Hello, World!"
```

```python
if __name__ == '__main__':

    with_parameters_no_return(1, 2, 3)

    result = with_parameters_with_return(1, 2, 3)
    print(f"Result: {result}")

    no_parameters_no_return()

    message = no_parameters_with_return()
    print(f"Message: {message}")
```

What Is Recursion?

- Recursion is a way of solving a problem by **solving a smaller version** of the same problem
- Think of a **Russian doll** as a *mental model* for recursion.

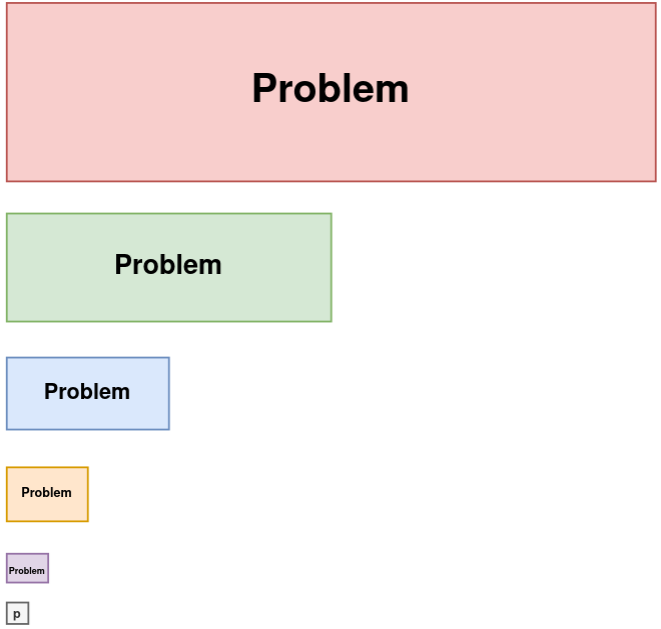
  Inside one doll is a smaller doll, and inside that is an even smaller one, until you reach the smallest doll



Recursion: *Type 1*

- Solve the problem by reducing it to a **single smaller problem**
- Each step (recursive call) makes the **problem smaller**
- The process stops at the smallest possible case, the **base case**

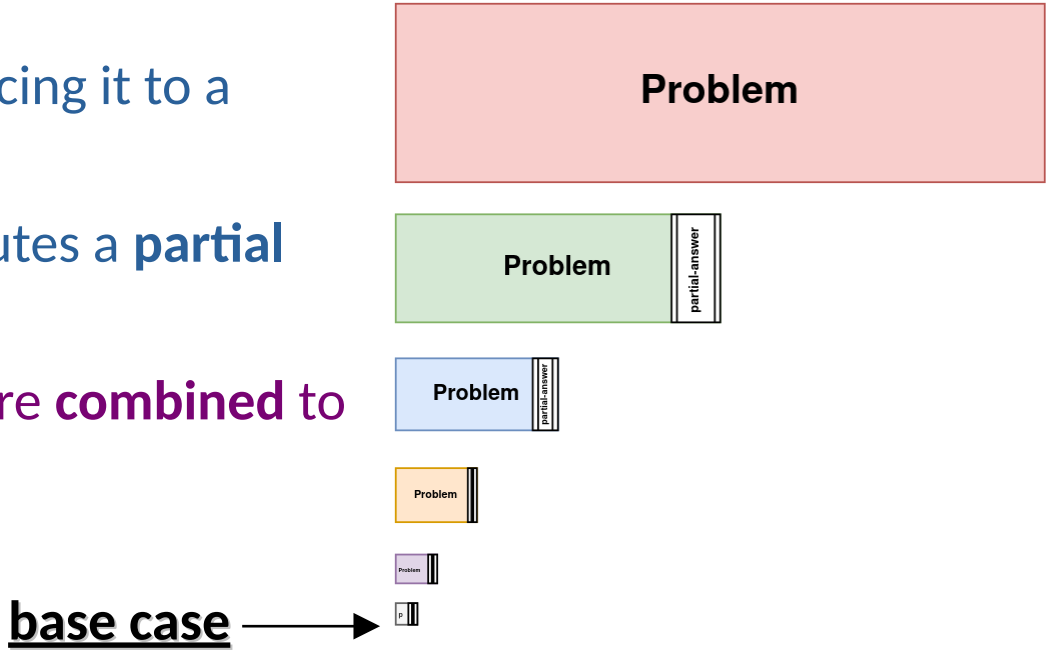
One recursive call
No partial answer



Recursion: *Type 2*

- Solve the problem by reducing it to a **single smaller problem**
- Each recursive call contributes a **partial answer**
- Then the partial answers are **combined** to produce the final solution

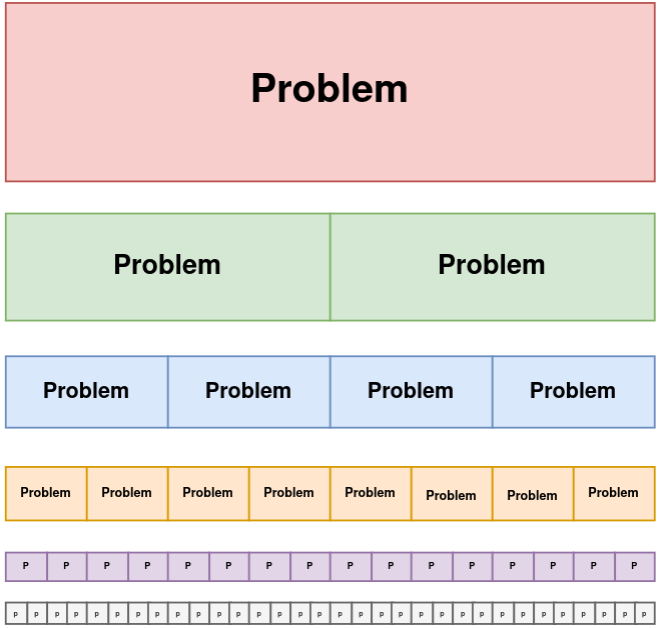
One recursive call
With a partial answer



Recursion: *Type 3*

- Solve the problem by breaking it into **<u>multiple</u> smaller problems**

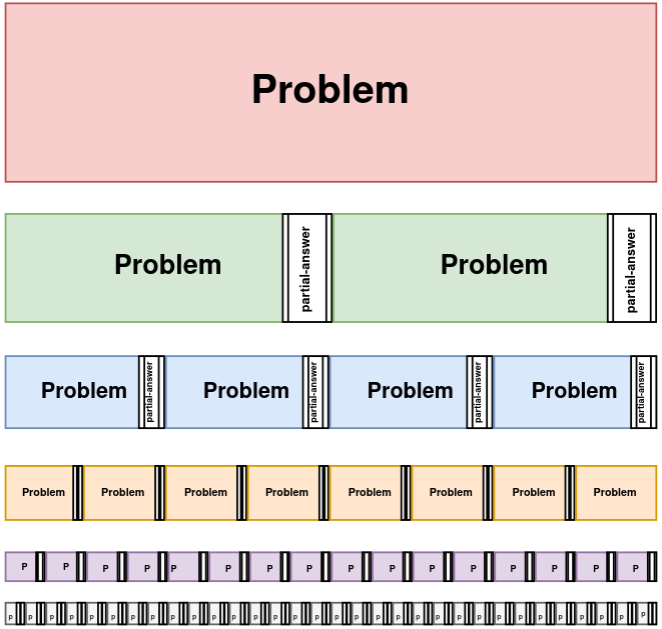
<u>Multiple</u> recursive calls,
<u>No</u> partial answer



Recursion: *Type 4*

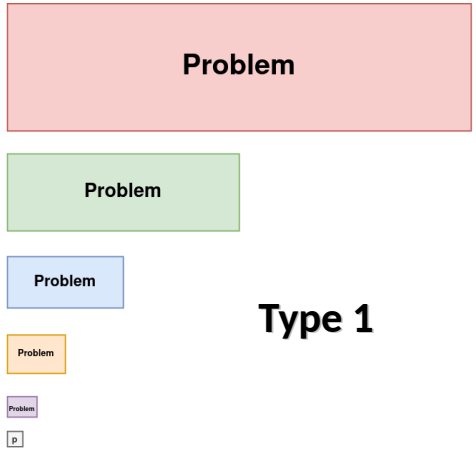
- Solve the problem by breaking it into **multiple smaller problems**
- Each recursive call contributes a **partial answer**

Multiple recursive calls, With partial answers

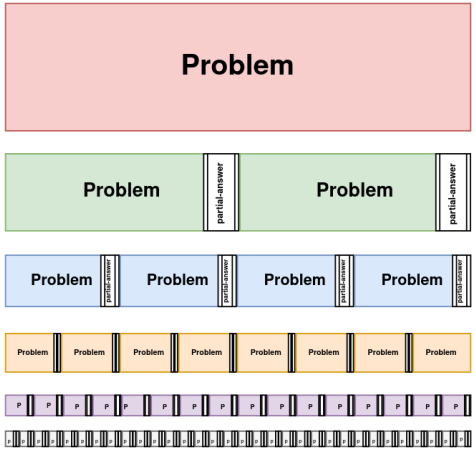


Comparing Types of Recursion

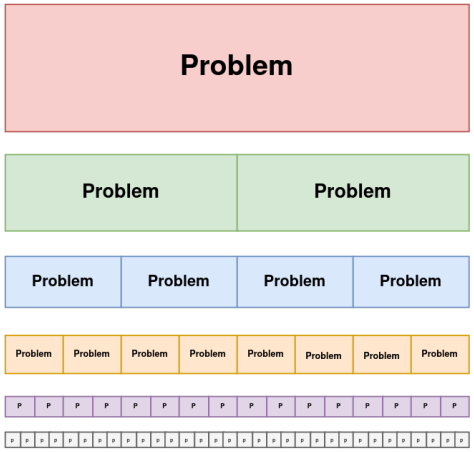
- Type 1: <u>One</u> recursive call — <u>No</u> partial answer
- Type 2: <u>One</u> recursive call — <u>With</u> a partial answer
- Type 3: <u>Multiple</u> recursive calls — <u>No</u> partial answer
- Type 4: <u>Multiple</u> recursive calls — <u>With</u> partial answers



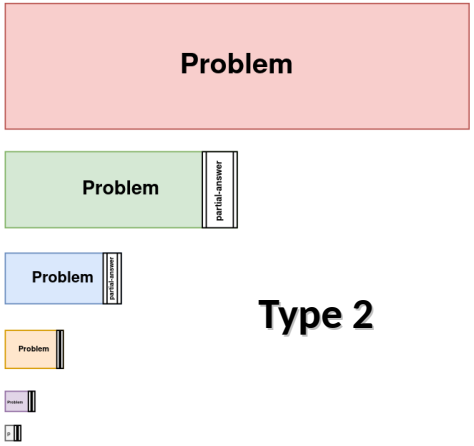
Type 1



Type 4



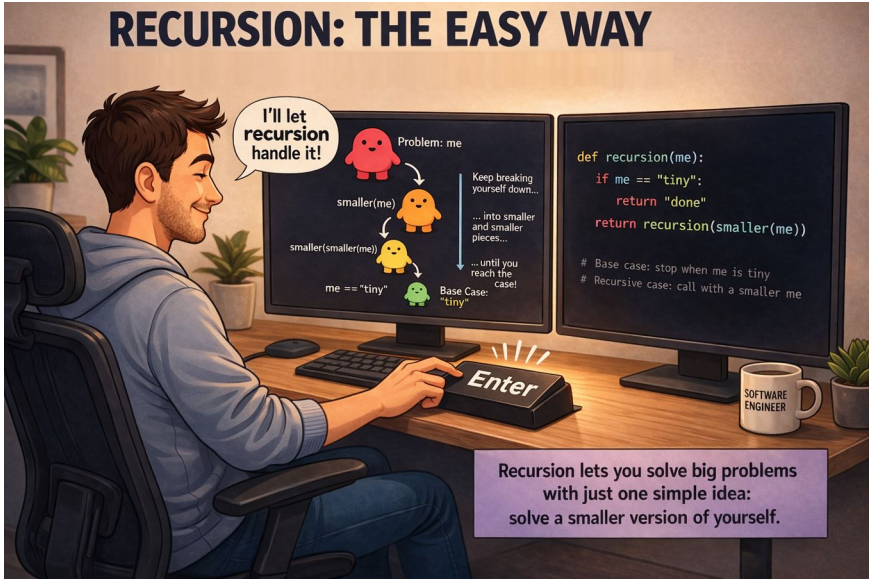
Type 3



Type 2

Why Do We Need Recursion?

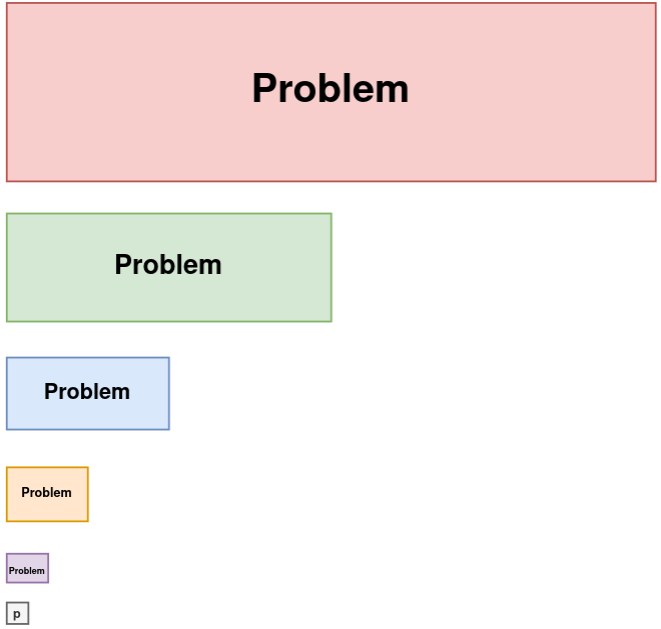
- **Some** problems are **easier** to solve with recursion
- Recursion helps us write solutions that are **organized** and **easier** to understand



Think Recursively: *Key Idea of Type 1*

- **One recursive** call at each step
- **No partial answer** is kept
- The goal is usually to **finish a process**

```
function solve(problem):
    # base case
    if problem is smallest:
        return answer

    # recursive case
    do_one_step()

    return solve(smaller_problem)
```



One recursive call
No partial answer

Think Recursively: *Count Down*

- Print the current number
- Count down the **remaining numbers**
- **Stop** when you reach zero

10, 9, 8, 7, … , 1

```
function countdown(n):
    # base case
    if n == 0:
        return

    # recursive case
    print(n)

    return countdown(n - 1)
```

Think Recursively: *Climb Stairs*

- Take one step
- Climb the remaining stairs
- **Stop** when no stairs are left

```
function climb(stairs):
    # base case
    if stairs == 0:
        return

    # recursive case
    take_one_step()

    return climb(stairs - 1)
```

Think Recursively: *Empty a Basket*

- Remove one item
- Empty the rest
- **Stop** when the basket is empty

```
function empty_basket(items):
    # base case
    if items == 0:
        return

    # recursive case
    remove_one_item()

    return empty_basket(items - 1)
```

Challenge Time!

Challenge 1: *Count down*

Problem: Starting from a number and going backward by 1 until you reach 0.

Your goal: Implement `countdown(n)`, a recursive function that counts down to 1.

Key idea:

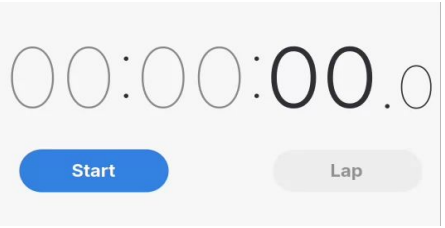
- **Base case:** when `n == 0`, stop and return
- **Recursive case:** print `n`, then call `countdown(n - 1)`

```
print(---)
return countdown(------)
```

Examples `countdown(2)`:

```
2
1
```

Challenge Time!



Challenge 1: *Count down*

Problem: Starting from a number and going backward by 1 until you reach 0.

Your goal: Implement `countdown(n)`, a recursive function that counts down to 1.

Key idea:

- **Base case:** when `n == 0`, stop and return
- **Recursive case:** print `n`, then call `countdown(n - 1)`

```
print(---)
return countdown(------)
```

Examples `countdown(2)`: 2

1

Challenge Time!

Challenge 1 Solved!

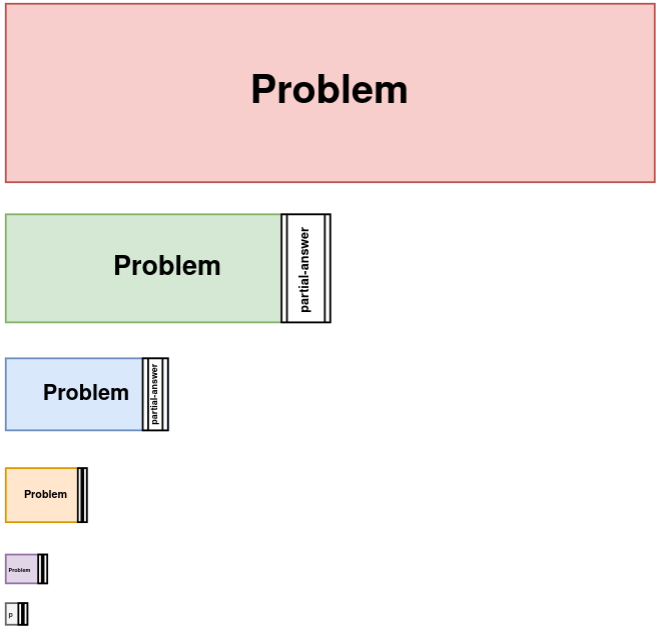
```python
def countdown(n):

    if n == 0:
        return

    print(n)

    return countdown(n - 1)

if __name__ == "__main__":

    countdown(5)
```

```
Output:
5
4
3
2
1
```

Think Recursively: *Key Idea of Type 2*

- **One** recursive call at each step
- Each level contributes **part of the solution**
- The final answer is built by **combining** these partial results

```
function solve(problem):
    # base case
    if problem is smallest:
        return base_answer

    # recursive case
    return partial_answer + solve(smaller_problem)
```



One recursive call
With a partial answer

Think Recursively: *Sum Numbers from 1 to n*

- Keep the current number
- Add it to the sum of the smaller numbers
- **Stop** at 1 and return 1

```
sum(4) = 4 + sum(3)
sum(3) = 3 + sum(2)
sum(2) = 2 + sum(1)
sum(1) = 1

sum(4) = 4 + (3 + (2 + 1))
```

```
function sum(n):
    # base case
    if n == 1:
        return 1

    # recursive case
    return n + sum(n - 1)
```

Think Recursively: *Factorial*

- Keep the current number
- Multiply it by the factorial of the smaller number
- **Stop** at 1 and return 1

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1

factorial(4) = 4 * (3 * (2 * 1))
```

```
function factorial(n):
    # base case
    if n == 1:
        return 1

    # recursive case
    return n * factorial(n - 1)
```

Think Recursively: *String of Stars*

- Keep one star
- Add it to the smaller result
- **Stop** at zero and return “”

```
function stars(n):
    # base case
    if n == 0:
        return ""

    # recursive case
    return "*" + stars(n - 1)
```

```
stars(3) = '*' + stars(2)
stars(2) = '*' + stars(1)
stars(1) = '*' + stars(0)
stars(0) = ''
stars(3) = '*' + ('*' + ('*' + '')) -> '***'
```

Challenge Time!

Challenge 2: *Factorial*

Problem: ***Factorial*** is a mathematical operation that means multiplying all the whole numbers from 1 up to a given number. For example, the factorial of 3 is 3 × 2 × 1 = 6.

Your goal: Implement `factorial(n)`, a recursive function that calculates n!.

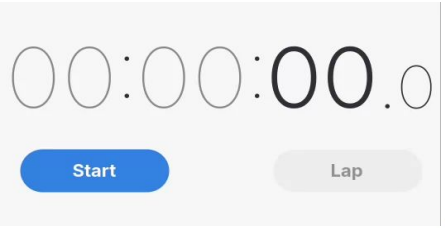
Key idea:

- **Base case:** `factorial(1) = 1`
- **Recursive case:** `factorial(n) = n * factorial(n - 1)`

Examples: `factorial(3) = 3 * 2 * 1 = 6`

`return ----------`

Challenge Time!



Challenge 2: *Factorial*

Problem: ***Factorial*** is a mathematical operation that means multiplying all the whole numbers from 1 up to a given number. For example, the factorial of 3 is 3 × 2 × 1 = 6.

Your goal: Implement `factorial(n)`, a recursive function that calculates n!.

Key idea:

- **Base case:** `factorial(1) = 1`
- **Recursive case:** `factorial(n) = n * factorial(n - 1)`

Examples: `factorial(3) = 3 * 2 * 1 = 6`

`return ----------`

Challenge Time!

Challenge 2 Solved!

```python
def factorial(n):
    # Base case
    if n == 1:
        return 1

    # Recursive case

    return n * factorial(n - 1)



if __name__ == "__main__":
    print(f"Factorial of 3: {factorial(3)}")
```

Output:

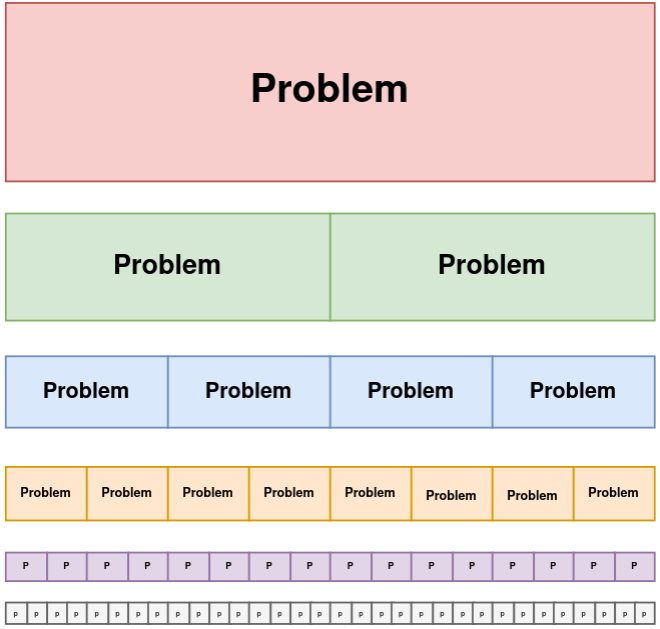
Factorial of 3: 6

Think Recursively: *Key Idea of Type 3*

- **Multiple** recursive calls
- **No partial answer** kept at each level
- Often used for search or backtracking

```
function solve(problem):
    # base case
    if problem is smallest:
        return base_answer

    # recursive case
    for smaller_problem in smaller_problems:
        solve(smaller_problem)
```

```
function solve(problem):
    # base case
    if problem is smallest:
        return base_answer

    # recursive case
    solve(smaller_problem1)
    solve(smaller_problem2)
    solve(smaller_problem3)
    ...
```



Multiple recursive calls
No partial answer

Think Recursively: *Search for a File in Folders*

- Search the **current folder**
- Search each **subfolder**
- **Stop** when the file is found or there are no more folders to search

```
function find_file(folder, target):
    # base case
    if folder is empty:
        return false
    # another base case
    if target is in folder:
        return true

    # recursive case
    for each subfolder in folder:
        if find_file(subfolder, target):
            return true

    return false
```

Think Recursively: *Escape a Maze*

- Try several **possible moves**
- Each move creates a **smaller search problem**
- **Stop** when can not move or found the exit
-

```
S = start
E = exit
# = blocked
blank = open cell
```

```
+---+---+---+---+---+---+
| S |   | # |   |   |   |
+---+---+---+---+---+---+
| # |   | # |   | # |   |
+---+---+---+---+---+---+
|   |   |   |   | # | E |
+---+---+---+---+---+---+
|   | # | # |   | # | # |
+---+---+---+---+---+---+
|   |   |   |   |   |   |
+---+---+---+---+---+---+
```

```
function can_escape(cell):
    # base case
    if cell is the exit:
        return true

    # another base case
    if cell is blocked or already visited:
        return false

    # recursive case
    mark_cell_as_visited()

    return can_escape(left) or
           can_escape(right) or
           can_escape(up) or
           can_escape(down)
```

Think Recursively: *Search a Family Tree*

- Check the current person
- Search each **child branch**
- **Stop** when the person is found or there are no more branches to search

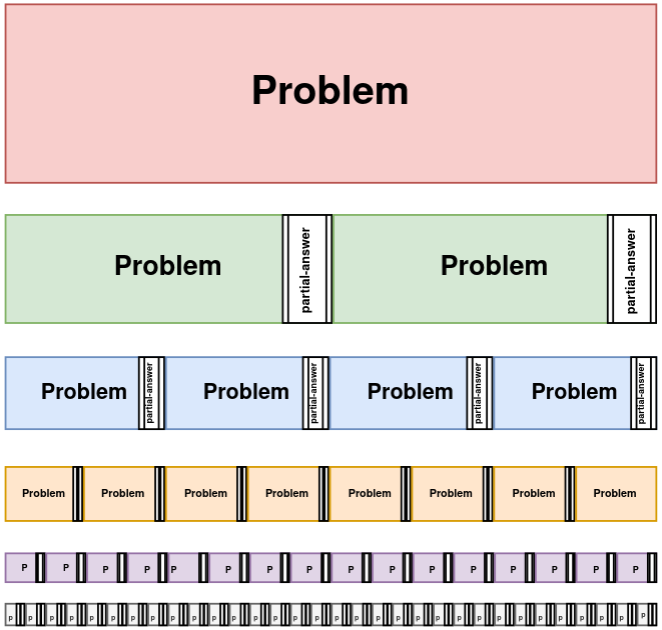
```
function find_person(person, target):
    # base case
    if person is empty:
        return false
    # another base case
    if person == target:
        return true

    # recursive case
    for each child in person.children:
        if find_person(child, target):
            return true

    return false
```

```
Family tree

          Alice
        /   |    \
     Ben   Cara   Dan
    /  \      \      \
  Eva  Finn    Gina   Hugo
             /   \
           Ivy   Jack
```

Think Recursively: *Key Idea of Type 4*

- **Multiple** recursive calls
- Each level contributes **part of the answer**
- Great for divide-and-combine problems

```
function solve(problem):
    # base case
    if problem is smallest:
        return base_answer

    # recursive case
    partial_answer = get_partial_answer(problem)
    return combine(partial_answer,
                   solve(smaller_problem1),
                   solve(smaller_problem2),
                   ...)
```



Multiple recursive calls
With partial answers

Think Recursively: *Count All Files in Folders*

- Count files in the **current folder**
- Count files in each **subfolder**
- **Stop** if folder is empty and return 0
- Add all counts together

```
function count_files(folder):
    # base case
    if folder is empty:
        return 0

    # recursive case
    total = files_in(folder)

    for each subfolder in folder:
        total = total + count_files(subfolder)

    return total
```

When to Avoid Recursion

DON’T use recursion when:

- A simple **loop is easier** to understand
- Solving the problem requires **many recursive calls**
- The recursive version is **harder to debug** and explain

When to Avoid Recursion

Avoid recursion for simple tasks such as:

- summing a list
- finding the maximum element
- scanning an array
- counting items

When to Use Recursion

Use recursion when:

- the problem **naturally breaks into smaller** subproblems
- the data has a **nested structure** (folders inside folders)
- the recursive solution is **easier** to read than the iterative one

When to Use Recursion

Use recursion, examples:

- searching folders and subfolders
- solving backtracking problems
- working with trees

```
        A
      /   \
     B     C
    / \     \
   D   E     F
      / \
     G   H
```

```python
def height(node):
    if node is None:
        return 0
    return 1 + max(height(node.left), height(node.right))
```

How to Use Recursion

Ask these three questions:

1) Is recursion better than loops here?

2) When does the function stop?

3) How does the problem get smaller?

```
def solve(problem):
    if base_case(problem):
        return answer
    smaller_problem = reduce(problem)
    return combine(solve(smaller_problem))
```

Preparing for the Final Challenge

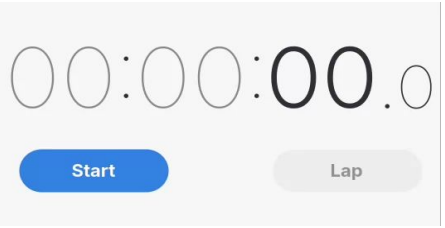
Organize Your Mind!

Write at least 50 words about what you learned today. Feel free to include a short summary of the class for your own reference.

Preparing for the Final Challenge

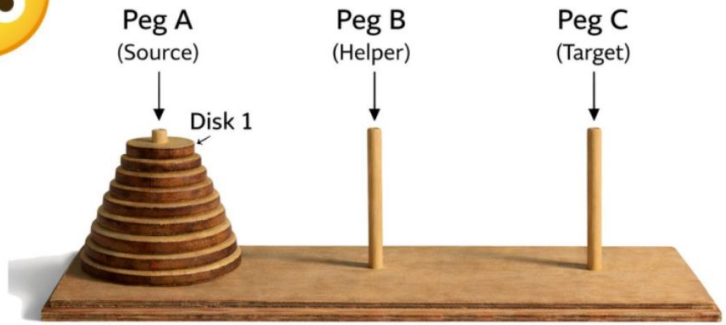
Organize Your Mind!

Write at least 50 words about what you learned today. Feel free to include a short summary of the class for your own reference.



Challenge Time!

Challenge 3: *Tower of Hanoi*



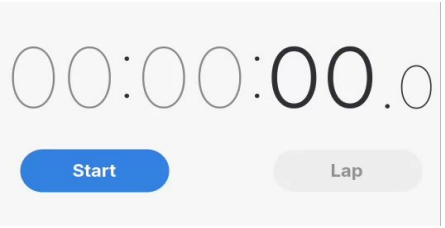
Problem: Move all disks from Peg A to Peg C

Rules:

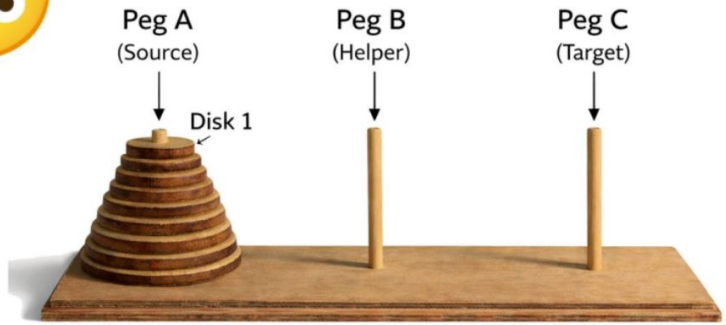
1. Move only one disk at a time
2. You may only move the top disk from a peg
3. Never place a larger disk on top of a smaller disk

Your goal: Write a recursive function that moves **n** disks from the source peg to the target peg

Challenge Time!



Challenge 3: *Tower of Hanoi*



Problem: Move all disks from Peg A to Peg C

Rules:

1. Move only one disk at a time
2. You may only move the top disk from a peg
3. Never place a larger disk on top of a smaller disk

Your goal: Write a recursive function that moves **n** disks from the source peg to the target peg

Challenge Time!

Challenge 3 Solved!

```python
def hanoi(n, source, target, helper):
    # Base case
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return

    # Recursive case
    hanoi(n - 1, source, helper, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n - 1, helper, target, source)

if __name__ == "__main__":
    num_disks = 3
    hanoi(num_disks, 'A', 'C', 'B')
```

Challenge Time!

```
Output:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Questions?

Thank you for your attention!